
INTERNAL · CONFIDENTIAL

TECHNICAL SPECIFICATION

The Analytical Engine

What the engine must do, and how well — the verifiable answer to the Analytical Backbone brief, stated above the stack.

Prepared for	Executive Steering Committee
Prepared by	Solution Engagement — Consulting
In response to	BRD · Analytical Backbone Initiative
Document type	Technical specification — for review & sign-off
Status	Draft 1 · consensus object, circulated before any build

This document specifies **what the analytical engine must do and how well** — every requirement is a verifiable outcome tied to one of the client's KPIs. It deliberately specifies **no database, framework, or architecture**; those are chosen in the solution-design phase and must answer to the targets set here.

1 The Problem, in Plain Language

One database is being asked to do two opposite jobs at once, and at the business's current volume those two jobs now fight for the same resources.

The platform runs roughly **\$500M in annual GMV** across **50,000 orders a day** (75,000 at peak). A single operational database serves both the storefront and the analytics that the business runs on. The storefront needs thousands of small, fast writes with predictable response every second. Analytics needs to scan large spans of history without a deadline on each query. These are opposite needs — and sharing one engine, they actively interfere.

The collision lands in two directions. A heavy report run during peak traffic locks resources the storefront needs, **degrading order processing exactly when volume is highest**. In the other direction, the transactional write load pushes reporting onto a 24-hour batch cadence, so **decisions are made on yesterday's numbers**. A core analytical query takes **12 minutes** where the business needs it in **under 5 seconds** — 144× too slow. That gap is not a query to be tuned; it is the signature of the wrong engine for the job.

THE STRUCTURAL POINT

You cannot tune your way out of two workloads with opposite needs sharing one engine. The durable fix is to give analytics its own lane — a dedicated analytical engine, fed from the transactional system but no longer contending with it. **This document specifies that engine by its required behavior, not its parts.**

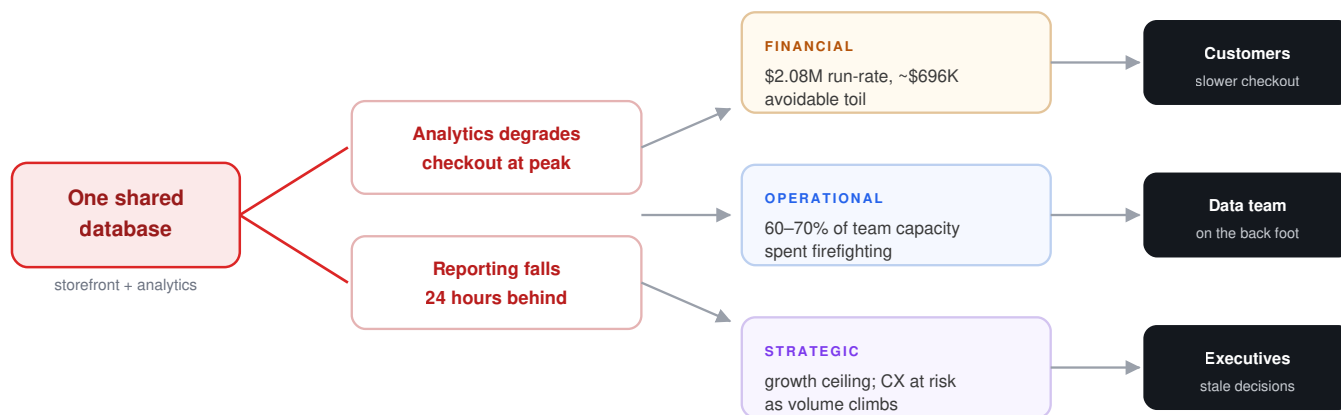


FIGURE 1 — HOW THE PAIN FLOWS: ONE SHARED ENGINE → TWO COLLISIONS → THREE BUSINESS COSTS → THE PEOPLE WHO FEEL THEM

2 Current State vs. Desired State

The required outcome is structural separation — analytics gets its own lane, fed from the transactional system of record but no longer competing with it.

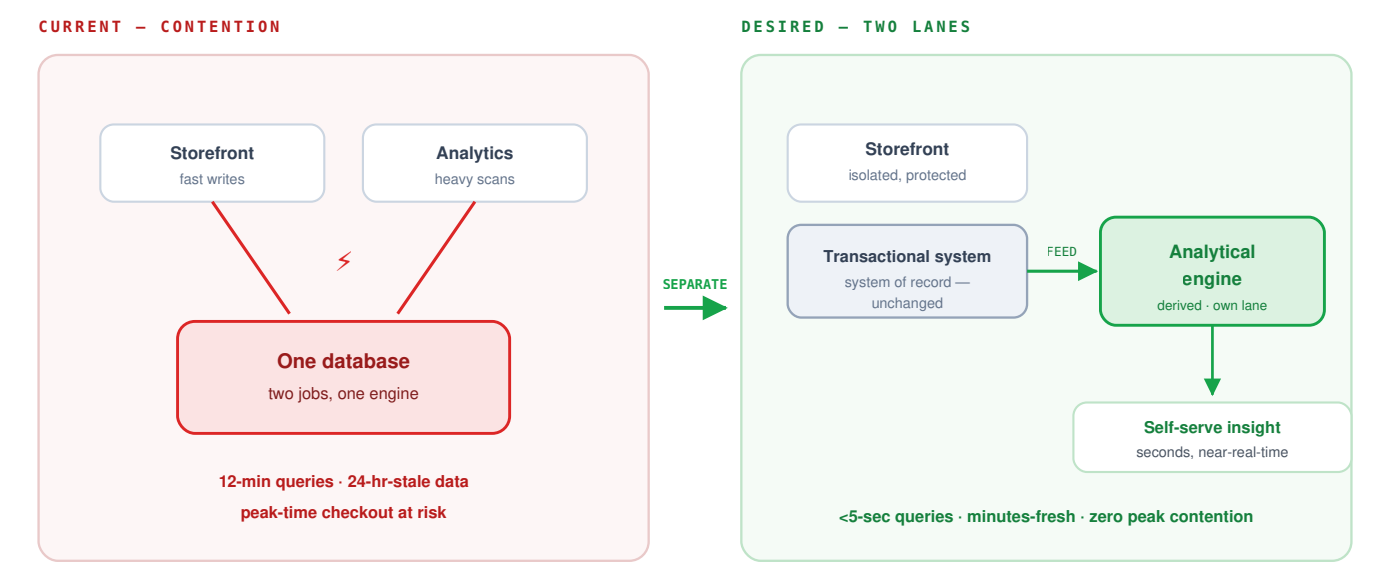


FIGURE 2 — CONCEPTUAL ONLY. THE ANALYTICAL ENGINE IS DERIVED FROM THE TRANSACTIONAL SYSTEM, WHICH STAYS THE SYSTEM OF RECORD. NO TECHNOLOGY IS IMPLIED.

WHAT THIS DIAGRAM DOES AND DOES NOT SAY

It commits to a **shape** — a separate analytical lane fed one-way from an unchanged source of truth — because that shape is what the brief requires. It does **not** commit to any database, processing framework, or serving interface. Those decisions are made in solution design and are accountable to the requirements and targets in Sections 4–5.

3 Scope — In and Out

This phase proves the separation and reclaims the operational headroom. Adjacent opportunities depend on this foundation and are deferred so this phase stays verifiable.

In scope

- Separating the analytical workload from the transactional database.
- A dependable, fast analytical store the business can query directly.
- Keeping analytical data current to within minutes of the source.
- Self-serve access to a defined set of analytical questions.
- Reducing the team's maintenance and incident burden.
- Capturing a transactional-latency baseline before any cutover (so isolation is provable).

Out of scope (this phase)

- Personalization / recommendation engines.
- Real-time machine-learning use cases.
- Replacing the storefront or its transactional database.
- Changes to order, payment, or fulfillment business logic.
- Net-new data sources beyond the current operational system.
- Selection of specific databases, frameworks, or architecture (solution-design phase).

Who is affected, and how relief reaches them

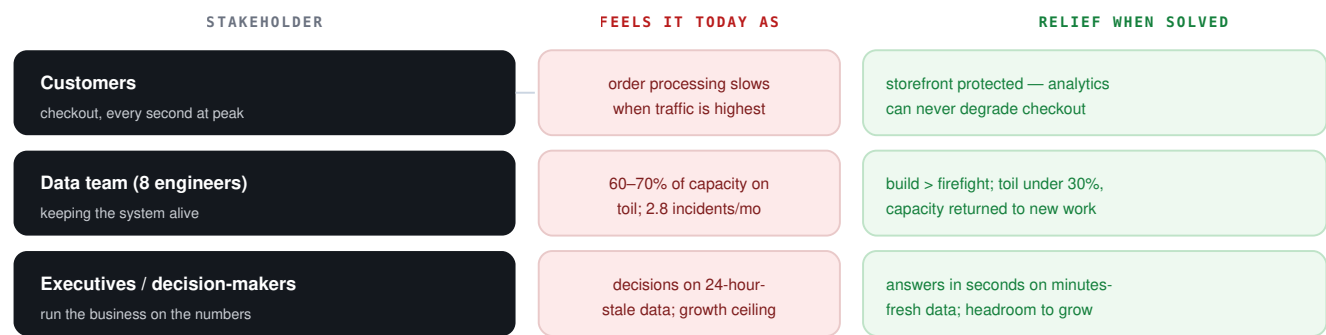


FIGURE 3 — THREE STAKEHOLDERS, THREE DISTINCT PAINS, THREE MEASURABLE RELIEFS

4 Requirements & Acceptance Criteria

Each requirement states what the engine must do and how well — a future test could pass or fail it — and each ties to one client KPI as current → target. *No technology is prescribed; the solution chooses how.*

E1 • WORKLOAD ISOLATION

Analytics must not degrade the storefront.

Under peak load (75,000 orders/day), analytical activity causes **zero measurable degradation** to transactional latency.

Verify: with a transactional-latency baseline captured first, run the heaviest analytical workload during a simulated/observed peak; transactional p95/p99 stay within the baseline's normal variance. [KPI • peak isolation](#)

E2 • QUERY RESPONSIVENESS

Analytical questions return fast enough to act on.

Typical analytical queries return in seconds, not minutes; worst-case stays bounded.

Verify: measured over the defined question set — p95 12 min → < 5 sec; p99 28 min → < 15 sec.

[KPI • query latency](#)

E3 • DATA FRESHNESS

Reporting reflects recent reality, not yesterday.

The lag from a business event to its appearance in analytics is minutes, not hours.

Verify: instrument event-to-reportable lag — 24 hr → near-real-time. Target window to be fixed in design (consultant default: ≤ 5 minutes, p95). [KPI • freshness](#)

E4 • SELF-SERVE ACCESS

Business users can ask questions without engineering.

A defined, frozen set of analytical questions is answerable directly by non-engineers, with no new report build each time.

Verify: a named list of questions (consultant default: the **10–15 highest-volume report requests** from the current queue) each returns a correct answer through self-serve, unaided. [KPI • self-serve set](#)

E5 · OPERATIONAL TOIL

The team builds more than it firefights.

Maintenance and incident response fall from the majority of capacity to a minority; contention-driven incidents become rare and recover fast.

Verify: tracked over a quarter — toil share 60–70% → < 30%; incidents 2.8/mo → < 0.5/mo; MTTR 5.2 hr → 30 min.

KPI · toil & incidents

E6 · AVAILABILITY

The analytical service is dependable.

The analytical engine meets a defined uptime target on a rolling basis.

Verify: measured availability 97.8% → 99.5% over a rolling 30-day window.

KPI · availability

5 Success Metrics

Success is movement on these KPIs, each stated current → target. The initiative is delivered when they are met — not when the work feels finished.

OUTCOME	CURRENT	TARGET	REQUIREMENT
Query latency — p95 (typical)	12 min	< 5 sec	E2
Query latency — p99 (worst case)	28 min	< 15 sec	E2
Data freshness (event → reportable)	24 hr	near-real-time	E3
Analytical availability	97.8%	99.5%	E6
Incident MTTR	5.2 hr	30 min	E5
Incident frequency (contention)	2.8 / mo	< 0.5 / mo	E5
Team capacity on toil	60–70%	< 30%	E5
Peak transactional degradation	present	zero measurable	E1

THE ACCEPTANCE PRINCIPLE

Every row above is a pre-registered test. Solution design is free to choose any technology, provided the built system turns this scorecard green — every line, measured, with the transactional-latency baseline captured before cutover so E1 is provable rather than asserted.

6 Open Assumptions to Resolve

These carry forward from the brief's constraints and four points that must be pinned before design. Each has a working default so the engagement keeps moving, and a named owner for the real answer.

Freshness target — set the number

"Near-real-time" must become a specific window, because it drives the feed mechanism and trades directly against E1 (continuously reading the source adds load to the very database we must not degrade).

Working default: $\leq$ 5-minute freshness (p95) — clears "minutes, not hours" with margin off the source; sub-minute deferred to a later phase.

OWNER · VP Data & Analytics (acceptable lag per use case) · CTO (source read-load tolerance)

The defined question set — freeze the list

E4 is untestable until "a defined set" is an actual list; it also shapes how the analytical data is modeled. Left open, scope has no finish line.

Working default: the 10–15 highest-volume questions from the current report-request queue, frozen as the acceptance set; everything else is phase two.

OWNER · VP Data & Analytics + business-unit leaders (own the question list)

Reclaimable toil — underwrite it honestly

The business case rests on recovering ~\$696K of toil, but a new analytical engine is itself a system to operate, and the "same team" constraint means eight people run two systems through transition — capacity dips before it improves.

Working default: underwrite ~\$400–450K reclaimable (some toil persists as new-system ops) and carry an explicit transition period in the plan.

OWNER · CFO (business-case figure) · VP Data (contention-driven vs. baseline-ops split)

Transactional baseline — capture it first

E1's "zero measurable degradation" is unprovable without a pre-cutover baseline of current transactional latency. It cannot be reconstructed after the fact.

Working default: make baseline instrumentation the first task of the engagement, before any build.

OWNER · CTO (instrumentation & baseline sign-off)

Carried constraints & assumptions (from the brief)

CONSTRAINT No storefront disruption Built and cut over without degrading the live storefront or its data.	CONSTRAINT Same team Delivered by the current data team of eight; no headcount increase assumed.	ASSUMPTION Source of truth unchanged Transactional DB stays the system of record; the analytical store is derived from it.
SIGN-OFF REQUESTED Approve this specification as the agreed statement of what the analytical engine must do and how well. On sign-off, the engagement proceeds to solution design — translating these requirements into an architecture and build plan that must answer to every target in Sections 4–5.		